# NES AI FINANCE & INSURANCE INTELLIGENCE PLATFORM

Implementation Roadmap & Milestones

**Document ID:** NES-PLAN-005 | **Version:** 1.0

**Build approach:** Cline-assisted development • Local-first Windows application

**Scope:** Personal research, paper trading and insurance document assistance

Incremental delivery, quality gates and risk management

## 1. Delivery strategy

Build the clickable prototype first, validate navigation and workflows, then integrate data and AI in separate milestones. Each phase must end with a runnable build and acceptance checks.

| Phase | Scope | Exit criteria |
|---|---|---|
| 1. Design foundation | Tokens, app shell, nav, responsive layout, shared controls. | Theme and navigation work at target widths. |
| 2. Clickable prototype | All primary pages, demo data, forms, states and Motion Lab. | No dead links; demo status visible; workflows are testable. |
| 3. Motion & 3D | 12 replayable animations, GSAP timelines, lightweight Three.js hero. | Reduced motion and no-WebGL fallback pass. |
| 4. Trading research | Provider adapter, screener, watchlists, stock analysis and historical data. | Source/freshness visible; data gaps handled. |
| 5. Backtest & paper | Assumptions, calculations, simulation ledger and risk limits. | Tests cover calculations; no live orders possible. |
| 6. Insurance | Local vault, extraction review, comparison, claims checklist and reminders. | Extracted values reviewable; source traceability present. |
| 7. Local AI | Grounded Copilot, local model integration and document Q&A.; | Outputs show limitations and relevant sources. |

| Phase | Scope | Exit criteria |
|---|---|---|
| 8. Hardening | Privacy, backups, accessibility, regression and documentation. | Release checklist complete. |

# 2. Work breakdown for each milestone

- Write scope and acceptance criteria before implementation.
- Create or update relevant typed contracts and data shapes.
- Implement the smallest vertical slice that can be run and inspected.
- Test normal, empty, invalid, failure and retry cases.
- Check responsive layout, keyboard interaction and reduced motion.
- Update changelog, known issues and relevant project documents.
- Do not start the next milestone with unresolved critical failures.

# 3. Definition of done

- Feature works end-to-end in the intended environment.
- No console errors or broken navigation in the changed flow.
- All buttons have meaningful actions or are clearly disabled with an explanation.
- Data source and demo/live status are truthful.
- Tests pass for affected logic.
- Accessibility and reduced-motion behaviour are checked.
- Documentation reflects the actual implementation.

# 4. Risks and mitigations

| Risk | Mitigation |
|---|---|
| Scope creep | Keep the prototype separate from real integrations; use milestones. |
| AI coding inconsistency | Maintain repository instructions, typed contracts and architecture decisions. |
| Misleading demo data | Label sample values and responses on every relevant page. |
| Animation performance | Lazy-load 3D, reduce motion and benchmark on actual hardware. |
| Incorrect backtests | Document fees, slippage, missing data and assumptions; write tests. |
| Sensitive document exposure | Use synthetic documents during prototype and define local privacy controls first. |

# 5. Suggested weekly review

- Review screenshots and actual working pages, not just agent summaries.
- Run the app from a clean start and test the critical path.
- Review changed files and dependency additions.
- Record decisions in docs/adr/ when a major technical choice is made.
- Update the milestone status and known-issues list.